

JVM Deep Dive – Manager Level (2+ Years Experience)

1. JVM, JRE, JDK – Architecture and Evolution

JVM (Java Virtual Machine):

The JVM is an abstract runtime engine responsible for executing Java bytecode.

It provides memory management, garbage collection, JIT compilation, threading, and security sandboxing.

JRE (Java Runtime Environment):

Contains JVM + core libraries required to run Java applications.

JDK (Java Development Kit):

Contains JRE + development tools (javac, jmap, jstack, jconsole, jfr).

Version Evolution:

- Java 7: PermGen present
- Java 8: Metaspace introduced (PermGen removed)
- Java 9+: Modular JDK (Project Jigsaw)
- Java 11: LTS, G1GC default
- Java 17+: ZGC, Shenandoah production-ready

2. JVM Architecture – Components and Flow

Major Components:

1. Class Loader Subsystem (Bootstrap, Platform, Application)
2. Runtime Data Areas
3. Execution Engine (Interpreter + JIT)
4. JNI Interface

Why Class Loader isolation matters:

Prevents class conflicts in app servers and microservices.

Execution Engine:

- Interpreter: Executes bytecode line-by-line
- JIT Compiler: Compiles hot methods into native code
- Code Cache: Stores compiled machine code

Pros:

- + Platform independence
- + Automatic memory management

Cons:

- GC pauses
- Warmup time (JIT compilation)

3. Runtime Data Areas – Shared vs Thread Specific

Shared:

- Heap (Young + Old Generation)
- Metaspace (Class metadata in native memory)
- Code Cache

Thread-Specific:

- Java Stack
- PC Register
- Native Method Stack

Impact of Thread Creation:

Each thread allocates its own stack (default ~1MB).

Large thread count → High native memory consumption.

Hence thread pools are recommended.

4. Java Memory Model (JMM)

Defines rules for visibility, ordering, and atomicity across threads.

Guarantees:

- Atomicity (primitive reads/writes)
- Visibility (volatile, synchronized)
- Ordering (happens-before)

Without JMM:

CPU caching + instruction reordering causes data races.

Happens-Before Rules:

- Program order
- Monitor lock rule
- Volatile rule
- Thread start/join rule

5. Volatile, CAS, Atomic Operations

volatile:

Ensures visibility and prevents reordering using memory barriers.

Limitations:

Does not guarantee atomicity for compound operations.

Atomic Classes:

Use CAS (Compare-And-Swap) via CPU instructions.

Pros of CAS:

- + Lock-free
- + Better scalability

Cons:

- Busy spinning under high contention

6. GC Analysis & Production Troubleshooting

Problem: Frequent Full GC causing latency spikes.

Analysis Steps:

1. Enable GC logs (-Xlog:gc*)
2. Analyze allocation rate
3. Check Old Gen occupancy
4. Heap dump analysis (MAT)
5. Use JFR for profiling

Common Causes:

- Memory leak
- Small heap
- Excessive promotion
- Large objects (humongous)

Resolution:

- Increase heap (-Xmx)
- Tune G1 regions
- Fix object retention

7. JVM Tuning Parameters

Memory:

-Xms, -Xmx, -Xss
-XX:MaxMetaspaceSize

GC:

-XX:+UseG1GC
-XX:+UseZGC (Java 15+)
-XX:+UseShenandoahGC (Java 17+)

JIT:

-XX:+PrintCompilation
-XX:+UnlockDiagnosticVMOptions

Monitoring Tools:

- JFR
- JMC
- jmap
- jstack
- jcmd

8. PermGen vs Metaspace

PermGen (Java 7 and below):

- Part of Heap
- Fixed size
- Caused OutOfMemoryError: PermGen space

Metaspace (Java 8+):

- Native memory
- Grows dynamically
- Better class metadata management

Still possible:

OutOfMemoryError: Metaspace (if native memory exhausted)

Why removed?

- ClassLoader leaks common
- Fixed sizing problematic

9. GC Algorithms Across Java Versions

Serial GC – Small apps

Parallel GC – Throughput focused

CMS – Low pause (Deprecated)

G1GC – Default since Java 9+

ZGC – Ultra-low pause (Java 15+ stable)

Shenandoah – Low latency (Java 17+)

Trade-offs:

Throughput vs Latency vs Memory footprint